

Norges teknisk–naturvitenskapelige
universitet
Institutt for datateknologi og
informatikk

TDT4102 Prosedyre- og objektorientert programmering Vår 2026

Øving 8

Frist: 2026-03-08

Mål for denne øvingen:

- Lære om operatorer og interaksjon mellom klasser av ulike typer.
- Lære å implementere og bruke klasser.
- Lære å bruke enkle klasser for et enkelt grafisk brukergrensesnitt (GUI).
- Lære å bruke pekere
- Lære om `unique_ptr` og `shared_ptr`

Generelle krav

- Bruk de eksakte navn og spesifikasjoner gitt i oppgaven.
- Denne øvingen skal implementeres uten hjelp fra `std_lib_facilities.h`.
- 70% av øvingen må godkjennes for at den skal vurderes som bestått.
- Øvingen skal leveres på INGIInious.
- Benytt VS Code til å skrive, kompilere og kjøre kode.

Anbefalt lesestoff:

- Kapittel 7, 19.5.2 og 19.5.3 i PPP
- [Dokumentasjon](#) til `AnimationWindow` -> GUI.

Del 1: Introduksjon til smartpekere

Smartpekere er objekter som automatisk håndterer minne i C++. De fungerer som vanlige pekere, men med innebygd logikk for å frigjøre ressursene de peker på når de ikke lenger trengs. Dette reduserer risikoen for minnelekkasjer og "dangling pointers".

Siden bruk av smartpekere minker risikoen for minnelekkasjer, samt gjør håndteringen av pekere lettere, er de veldig ofte brukt i C++. I denne øvingen skal fokuset ligge på råpekere, slik at dere blir stødige på minnehåndteringen, men det er likevel lurt å være kjent med smartpekere og hvordan man bruker dem.

Nyttig å vite: `std::unique_ptr` og `std::shared_ptr`

Hva er en `std::unique_ptr`? `std::unique_ptr` er en smartpeker som eier og håndterer et annet objekt gjennom en peker, og sletter automatisk minnet som tilhører objektet når `std::unique_ptr`-instansen blir destruert. Når en `std::unique_ptr` blir opprettet kaller den automatisk `new`, og når den går ut av skop kaller den `delete`. Dette vil si at vi slipper å bruke `new` og `delete` når vi skal bruke dynamisk allokerede objekter. Merk at dette faget bare er en liten intro til det man kan gjøre med `std::unique_ptr`.

Hvordan opprette en `std::unique_ptr`? C++14-funksjonen `std::make_unique` brukes for å lage et nytt `std::unique_ptr`-objekt. Det allokerer også nødvendig minne til objektet `std::unique_ptr` håndterer.

```
#include <memory> //For unique_ptr
/* Her opprettes en unique_ptr som håndterer et Student-objekt.
Argumentene til funksjonen blir gitt direkte til Student-konstruktøren. */
std::unique_ptr<Student> s1 = std::make_unique<Student>("daso",
"daso@stud.ntnu.no");
/* Vi kan også bruke auto når vi oppretter en unique_ptr. */
auto s2 = std::make_unique<Student>("lana", "lana@stud.ntnu.no");
```

Hvordan bruke en `std::unique_ptr`? Derefereringsoperatoren (*) og piloperatoren (->) blir brukt som om det er en vanlig peker.

```
std::cout << s1->getName() << '\n';
std::cout << *s2 << '\n';
```

En `std::unique_ptr` eier objektet det peker til (bare en `std::unique_ptr` kan peke på et objekt om gangen) — den tillater ikke pekeren å bli kopiert. (Hvis vi hadde kunnet kopiere en `std::unique_ptr` og en av pekerne hadde gått ut av skop, ville objektet blitt slettet, og de gjenværende `std::unique_ptr` ville peke på minne vi ikke vet hva inneholder.) Derimot kan eierskapet overføres vha. `std::move`. Vi sier at `std::move` overfører eierskapet av objektet.

```
/* Overfører eigarskap frå s1 til s3. */
std::unique_ptr<Student> s3 = std::move(s1);
```

```
/* Verdien til s1 er nå udefinert. */
/* Kodelinjen under vil ikke kompilere, for std::unique_ptr kan ikke bli
kopiert*/
// auto s3 = s1;
```

Etter denne operasjonen er `s3` en `std::unique_ptr` som eier det `Student`-objektet som `s1` tidligere eide. `s1` er derimot i en «gyldig men uspesifisert» tilstand.

Noen ganger ønsker vi å la andre bruke objektene som blir pekt på av `std::unique_ptr`-instansen uten å overføre eierskapet. Vi kan da få tak i den underliggende pekeren, vha. medlemsfunksjonen `get`.

```
void printStudent(Student* sPtr);
printStudent(s2.get()); /* s2.get() returnerer den underliggende
pekeren.*/
```

Medlemsfunksjonen `get` kan også bli brukt til å sjekke om en `std::unique_ptr` har et tilknyttet objekt, ved å sammenlikne med `nullptr`.

```
if (s1.get() != nullptr) {
std::cout << "S1 contains an object\n";
} else {
std::cout << "S1 does not contain an object\n"; // <- Dette skrives ut
}
if (s3.get() != nullptr) {
std::cout << "S3 contains an object\n"; // <- Dette skrives ut
} else {
std::cout << "S3 does not contain an object\n";
}
}
```

`std::unique_ptr` kan virke vanskelig, men ikke overkompliser det! Tenk på det som en vanlig peker, bare at den eier objektet den peker på og dermed har ansvar for å fiks minnet til objektet selv. Og siden det blir krøll hvis flere skal eie det samme objektet, kan man ikke ha mer enn én `std::unique_ptr` til et objekt, så dersom en annen `std::unique_ptr` skal peke dit må man «`std::move`» eigarskapet.

`std::shared_ptr` fungerer på mange måter likt, men objektet den peker på kan nå bli eid av flere `std::shared_ptr`s. For hver peker du da har til objektet vil en teller legge til én. Hvis pekerene går ut av skop, eller blir fjerna, vil telleren reduseres med én, og først når den siste `std::shared_ptr`-en går ut av skop, blir objektet slettet.

1 Smartpekere (5%)

Merk at vi i denne oppgaven skal fremprovosere noen feil, så programmet ditt kan og skal oppføre seg litt rart. Koden i denne oppgaven skal skrives/redigeres på i filene `xx` og `xx`.

- a) Gitt følgende kode, hva forventer du at skrives ut når `smartptrTest` kjører?
Du kan finne deklarasjonen av klassen `Book` i `smartptr.h`.

Denne oppgaven besvares i INGIInious.

smartptr.cpp

```
#include "smartptr.h"

void smartptrTest () {
    std::shared_ptr<Book> b1(new Book("1984", 328));
    std::shared_ptr<Book> b2(b1);

    std::unique_ptr<Book> b3(new Book("Brave New World", 288));
    std::unique_ptr<Book> b4(b3);

    std::shared_ptr<Book> b5(new Book("Fahrenheit 451", 194));
    Book* raw = b5.get();
    std::shared_ptr<Book> b6(raw);

    b1->read();
    b2->read();
    b3->read();
    b4->read();
    b5->read();
    b6->read();
}
```

- b) Fjern kommentarene rundt koden i `smartptr.cpp` og prøv å kjør koden. Skjer det du forventet skulle skje?

Denne oppgaven besvares i INGIInious.

- c) Regiger koden i `smartptrTest` slik at den bruker smartpekere riktig og kan kompileres uten feil.

Del 2: NTNU-samkjøring (55%)

Du har fått sommerjobb i det nye studentforetaket «EcoTrans» som er startet av noen miljøbevisste NTNU-studenter. Omorganiseringen til nye NTNU har skapt et behov for koordinert transport mellom byene Trondheim, Ålesund og Gjøvik. Hver uke reiser mange ansatte og studenter mellom disse byene, ofte i privatbiler med ledige seter. EcoTrans vil lage et enkelt datasystem for å stimulere til miljøvennlig samkjøring, og i denne oppgaven skal du skrive noen kodesnutter for et slikt system.

NB: I resten av denne øvingen skal du bruke råpekere med mindre en oppgave eksplisitt ber om bruk av smartpekere.

NB: I denne øvingen *skal* du bruke const-funksjoner der du kan.

Nyttig å vite: Const correctness

Det er god praksis å markere medlemsfunksjoner som ikke endrer objektet med `const`.

Dette gjør det enklere å finne feil i koden, og lar oss bruke medlemsfunksjonene selv om objektet er konstant.

```
class NumberClass {
    int number;
public:
    NumberClass(int number)
        : number{number} {}

    // markert const
    int getNumber() const {
        return number;
    }

    // ikke markert const
    void setNumber(int newNumber) {
        number = newNumber;
    }
}

int main () {
    NumberClass x{3};

    int i = x.getNumber(); // OK
    x.setNumber(i+1); // OK

    const NumberClass y(4);
    int j = y.getNumber(); // OK

    // IKKE OK!
    // Kompileringsfeil:
    // kan ikke kalle en funksjon
    // som ikke er markert const,
    // på et const objekt
    y.setNumber(j+1);
}
```

2 Car-klasse (10%)

a) Deklarer en klasse Car.

Car skal ha et heltall `freeSeats` som privat medlemsvariabel som indikerer hvor mange ledige seter det er i bilen. Car skal også ha to `public` medlemsfunksjoner `hasFreeSeats` og `reserveFreeSeat`. `hasFreeSeats` returnerer `true` om bilen har ledige seter, og `false` ellers. `reserveFreeSeat` «reserverer» et ledig sete ved å dekrementere `freeSeats`-variabelen (du kan gå ut i fra at funksjonen bare blir kalt på om det er ledige seter).

Deklarasjoner for medlemsfunksjonene:

```
bool hasFreeSeats() const;
void reserveFreeSeat();
```

NB: husk `const` correctness.

b) Deklarer og implementer en konstruktør som tar inn hvor mange ledige seter bilen har. Dersom du ønsker å bruke initialiseringsliste, *IKKE* bruk det i `.h`-filen fordi dette ødelegger for autoretteren. Du kan fint implementere det i `.cpp`-filen i stedet.

c) Implementer `hasFreeSeats()` og `reserveFreeSeat()`.

Husk at `hasFreeSeats()` må være deklart med `const` i `.h`-filen og i `.cpp`-filen for at kompilatoren skal skjønne at det er snakk om samme funksjon.

3 Person-klassen (20%)

a) Deklarer medlemsvariablene til klassen Person.

Denne skal ha de private medlemsvariablene `name` og `email`, begge av typen `string`. I tillegg skal klassen ha en medlemsvariabel `car` av datatypen `Car *`, som er en peker

til en `Car`. Legg merke til at vi ønsker å bruke en peker og ikke referanse til `Car`-objektet. Grunnen til at vi ønsker å bruke en peker er fordi en peker kan ha verdien `nullptr` og det passer fint for å representere at en person *ikke* har bil. Om vi bruker referanse istedenfor peker ville det vært vanskeligere å representere dette. Dette er godt forklart i læreboka §7.4, og mot slutten av det avsnittet er det oppsummert når man anbefaler pass-by-value, peker, eller referanse-parameter.

Klassen skal ha en konstruktør som setter `name`, `email` og `car` til verdier gitt av parameterlisten. For `car` bruker vi `nullptr` som et såkalt «default argument» (standard-verdi). Det betyr at konstruktøren kan brukes med bare de to første parametrene, og da vil den tredje få denne standard-verdien. Se nyttig-å-vite-boks om dette temaet.

Nyttig å vite: default arguments

For å unngå at man skal definere flere ulike funksjoner som gjør det samme, men har ulikt antall på parametere i parameterlisten, så finnes det default arguments. For eksempel kan en funksjon som alltid skal legger sammen to tall være standardisert til å inkrementere det første argumentet med en, dersom det andre argumentet ikke er oppgitt. Istedenfor å lage to funksjoner som gjør samme arbeid eller kaller på en annen, er det hensiktsmessig å kombinere de. Dette kan man også bruke med medlemsfunksjoner i klasser.

```
void Adder(int a, int b = 1);
//void Adder(int a = 1, int b); // Gir kompileringsfeil

int main() {
    Adder(42, 42); // a+b=84
    Adder(42); // a+b=43
}

void Adder(int a, int b) {
    cout << "a+b=" << a+b;
}
```

Default argument skrives i deklarasjonen, men ikke i definisjonen. Argumenter som har default-verdi må også skrives sist i parameterlisten.

b) Deklarer og implementer konstruktøren og get-/set-funksjoner.

Husk at deklarasjonen gjøres i `.h`-filen, mens implementeringen gjøres i `.cpp`-filen. I denne oppgaven skal du deklarere og implementere konstruktøren, og tre get- og set-funksjoner.

Konstruktøren til `Person` skal ta inn 3 variabler, `name`, `email` og `car` i denne rekkefølgen. Det er viktig at variablene kommer i denne rekkefølgen i parameterlisten, og at de har samme datatype som variablene de skal initialisere. Bruk gjerne initialiseringsliste i konstruktøren, men dette *må* gjøres i `.cpp`-filen.

Videre skal følgende get- og set-funksjoner deklarerer og implementeres:

- `Person::getEmail()` - returnerer medlemsvariabelen `email`
- `Person::getName()` - returnerer medlemsvariabelen `name`
- `Person::setEmail(const std::string& email)` - Tar inn en epost-adresse og tilordner verdien til medlemsvariabelen `email`

Husk å deklarerer medlemsfunksjonene som *ikke* endrer på medlemsvariablene til klassen som `const` slik at kompilatoren forstår dette. Da er det mulig å opprette const-instanser av klassen og kjøre disse medlemsfunksjonene uten at det oppstår problemer.

c) **Deklarer og implementer medlemsfunksjonen `hasAvailableSeats()`.**

Funksjonen returnerer `true` om personen eier en bil og bilen har ledige seter og `false` ellers.

NB: husk `const correctness`.

d) **Overlast `operator<<`, som skal skrive ut innholdet i `Person` til en `ostream`.
Drøft:**

- Hvorfor bør denne operatoren deklarerer med `const`-parameter? (t.d. `const Person& p`)
- Når bør vi, og når bør vi ikke (ev. kan ikke) bruke `const`-parameter?

e) **Deklarer og implementer destruktøren.**

Destruktøren til `Person` skal frigjøre minnet til `Car`-pekeren `car`. Når man driver med grunn kopiering av råpekere blir det fort vanskelig å holde styr på minnet vi har allokert. Da er det viktig å sette tydelige regler for hvilke klasser eier hvilket minne. I denne øvingen velger vi å la `Person`-klassen eie minnet til `Car`-pekeren den tar inn som parameter, og destruktøren til `Person` må derfor håndtere slettingen av `Car`-pekeren. En konsekvens av å la `Person` eie minnet til `Car`-pekeren er at det opprinnelige `Car`-objektet blir slettet når `Person` går ut av skop, og du må passe på selv at det objektet ikke brukes andre steder i koden. Det betyr altså at hver `Car` kun kan tilhøre *én* `Person`.

Den letteste måten å unngå å håndtere alt dette selv er å bruke smartpekere (f.eks. `std::unique_ptr`), men som sagt har vi valgt å holde oss til råpekere i denne oppgaven for å få en god forståelse av pekere og minneallokering.

f) **Skriv tester for `Person` i `main()`.**

Opprett flere personer, og prøv å test ulike tilfeller (t.d. har personen bil? Hva med når personen ikke har bil?).

4 Meeting-klassa (25%)

a) **Deklarer en `scoped enum`, med navn `Campus`, som innehold verdier for de ulike byene (Trondheim, Ålesund og Gjøvik).**

La deklarasjonen av `enum class Campus` ligge i `Meeting.h`. forkortelsene du skal bruke for hver av byene skal være `trh`, `aal` og `gjo` henholdsvis.

b) **Overlast `operator<<` for `Campus`, som skal skrive campusnavnet til en `ostream`.**
I `.h`-filen skal du deklarere overlastingen, mens i `.cpp`-filen skal du implementere overlastingen. Bruk det forhåndsdeklarererte `map-et` `campusToString` i `.h`-filen i oppgaven. (Dette er viktig for autorettingen sin del)

Husk: fjern kommentarene slik at du kan bruke den.

c) **Deklarer medlemsvariablene til klassen `Meeting`.**

Klassen skal ha følgende private medlemsvariabler:

```

int day;
int startTime;
int endTime;
Campus location;
std::string subject;
const Person* leader;
vector<Person*> participants;

```

d) Implementer get-funksjoner for medlemsvariablene

Alle variablene unntatt `participants` skal ha `get`-funksjoner. Fordi alle `get`-funksjonene kan implementeres i én linje, skal de ikke bare deklarerer, men også *implementeres* i `.h`-filen i klasse-definisjonen. F.eks.:

```
int getDay() const { return day; }
```

Navnet på funksjonene *skal* være:

```

getDay() getStartTime() getEndTime()
getLocation() getSubject() getLeader()

```

NB: husk `const correctness`.

e) Deklarer og implementer medlemsfunksjonen `addParticipant`.

Den skal ta inn en peker til et `Person` objekt og legge den inn i `participants`.
Husk: bruk nøyaktig samme funksjonsnavn som spesifisert i oppgaven.

f) Implementer konstruktøren for Meeting-klassen som er deklart i `.h`-filen.

Husk at møtelederen også er en deltaker.

g) Teori: Hvorfor har vi ikke implementert en destruktør for klassen `Meeting`?

Denne oppgaven besvares i INGIInious.

h) Deklarer og implementer funksjonen `getParticipantList()`.

Dette skal være en medlemsfunksjon i `Meeting`. Funksjonen har ingen parametere og returnerer en `std::vector<std::string>` med navn på deltakerene.

NB: husk `const correctness`.

i) Deklarer og implementer funksjonen `findPotentialCoDriving`.

Dette skal være en medlemsfunksjon i `Meeting`. Funksjonen skal ta inn et annet møte via **pass-by-reference**, og skal returnere en vektor med `Person`-pekere. Vektoren skal bestå av alle personer fra det andre møtet som:

- har ledige plasser i bilen sin og
- der det andre møtet
 - er på samme sted som `this`-møtet,
 - er på samme dag som `this`-møtet,
 - har start-tid som er mindre enn en time forskjellig fra start-tida til `this`-møtet, og
 - har slutt-tid som er mindre enn en time forskjellig fra slutt-tida til `this`-møtet.

NB: husk `const correctness`.

Hint: Funksjonen vil ha følgende returtype: `std::vector<Person*>`.

j) Deklarer og implementer overlastingen `operator<<` for `Meeting`.

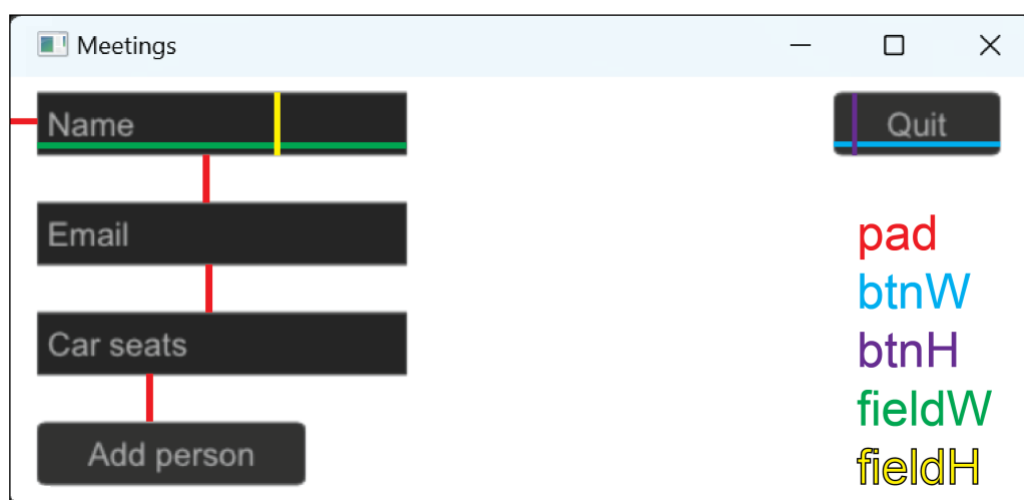
Denne operatoren skal IKKE være en `friend` av `Meeting`. Du står fritt til å velge format selv, men du skal skrive ut `subject`, `location`, `startTime`, `endTime`, og navnet på møtelederen. I tillegg skal den skrive ut en liste med navnet på alle deltakerene.

Test funksjonen din fra `main()`.

DEL 3: GUI for samkjøring og møteplanlegging (40%)

For å gjøre programmet mer brukervennlig vil dere lage et grafisk brukergrensesnitt (GUI) der passasjerer kan melde seg inn. Den skal ha to tekstfelt, et for å skrive inn passasjerens navn og et for e-post. Det skal også være to knapper: en for å legge til personen og den andre for å avslutte programmet.

Før du går løs på kodingen til et GUI, er det veldig lurt å danne seg en skisse for hvordan vinduet skal se ut. Det er også lurt å navngi alle ulike avstander i vinduet. Dette er delvis fordi man ikke trenger å sjonglere alle de ulike verdiene i hodet, men mest fordi man da kan endre på alle felles avstander på et sted. Under kommer en mulig skisse av GUI-et der hver farge er koblet til en variabel. Det er ikke et krav at du følger denne skissen, men alle elementer skal være med til slutt.



Alle variabler og funksjoner i resten av oppgavene skal være medlem av vinduet-klassen vi lager.

5 Oppretting av GUI (20%)

I denne oppgaven skal vi bruke `AnimationWindow` til å lage et vindu, der vi overlater programstyringa til selve vinduet. I praksis betyr dette at vi må lage vår egen vindu-klasse som arver fra `AnimationWindow`, og at klassen skal styre programmet.

Videre skal vi bruke widgetsene `TDT4102::TextInput` og `TDT4102::Button`, og man må inkludere `widgets/TextInput.h` og `widgets/Button.h` for å bruke de.

- a) Implementer konstruktøren `MeetingWindow(int x, int y, int w, int h, const string& title)` for klassen `MeetingWindow` i `MeetingWindow.cpp`.

For å opprette et vindu av ønsket størrelse, må du initialisere konstruktøren til `AnimationWindow` med ønsket posisjon, høyde og bredde. Dette kan du gjøre ved å kalle på `AnimationWindow`-konstruktøren i initialiseringslisten til `MeetingWindow`-klassen, der argumentene skal være de du tok inn i `MeetingWindow`-konstruktøren. Konstruktørkroppen står tom inntil videre.

b) Lag et MeetingWindow-objekt i main().

Lag et objekt av typen `MeetingWindow` i `main()`. Prøvekjør koden, du skal få opp et blankt vindu. *Tips: bruk `wait_for_close()` funksjonen for at vinduet skal være oppe fram til brukeren lukker det.*

- c) Før du begynner på å legge inn elementer, er det lurt sette inn noen heltall i klassen som definerer oppsettet i vinduet, se skissen over. Siden disse ikke skal endres og er definerte før kompileringen, skal du deklare de **static constexpr int** og gi dem verdier direkte i `.h`-filen. Bruk nøyaktig samme variabelnavn som i skissen i starten av oppgaven.

Nyttig å vite: static og static constexpr

En statisk medlemsvariabel er en variabel som er felles for alle instansene av klassen. Man kan skriva `static` før medlemsvariabelen for å sikre seg at det bare finnes en kopi av variabelen når programmet blir kjørt, istedenfor en kopi per objekt som lages i klassen.

`inline` trengs for å kunne initialisere statiske variabler i klassedefinisjonen. Å deklare en variabel som `constexpr` gir kompilatoren beskjed om at variabelen skal evalueres ved kompilering. Det gir oss også muligheten til å bruke variabelen i `constexpr`-funksjonar som kan evalueres ved kompilering. En variabel som er `static constexpr` er derfor felles for alle instansene av klassen, og evalueres ved kompilering. Dette betyr også at den kan (og må) initialiseres direkte i `.h`-filen til klassen. På denne måten oppnår man raskere kjøring av koden, fordi den ene kopien av variabelen allerede er evaluert ved kompilering og trengs aldri å evalueres igjen.

Nyttig å vite: Kort om callback

En callback-funksjon er en funksjon blir kalt når du trykker på en knapp i GUI-et. Den skal ikke ta inn noen parametere og returnerer `void`. For å koble en callback-funksjon til en knapp bruker vi `setCallback()`-funksjonen til knappen. Les mer om callback-funksjoner [her](#).

- d) **Deklarer og implementer callback-funksjonen `void cb_quit()`.** Denne callback-funksjonen skal brukes av avslutnings-knappen, så derfor må den kalle på den arvede medlemsfunksjonen `close()` som lukker vinduet.
- e) **Legg inn et `TDT4102::Button`-objekt, `quitBtn`, som medlemsvariabel og konstruer `quitBtn` i initialiseringslista til `MeetingWindow`.**
Knappen `quitBtn` skal deklarerer i `MeetingWindow.h` og konstrueres (initialiseres) i initialiseringslisten i `MeetingWindow.cpp`.
- f) **Legg til `quitBtn` i GUI.**
`quitBtn` legges til vinduet med `add(quitBtn)` i funksjonskroppen til `MeetingWindow`-konstruktøren.
- g) **Legg til callback-funksjon til `quitBtn`.**
`quitBtn` skal bruke `cb_quit` som callback-funksjon, som gjøres slik `quitBtn.setCallback(std::bind(&MeetingWindow::cb_quit, this));`. Prøv å kjør programmet og se om det fungerer som forventet.

6 Person-funksjonalitet (20%)

I denne oppgaven skal vi legge til et par widgets til vinduet. Her er det viktig å passe på at ingen av widgets-ene ligger oppå hverandre. Da vil de ikke oppføre seg som forventet.

a) **Legg til to `TDT4102::TextInput` `personName` og `personEmail` i GUI.**

Disse er to innskrivingsfelt for parameterene til en ny `Person`. Deklarer variablene i `MeetingWindow.h`, og initialiser de i `MeetingWindow.cpp`. Innskrivingsfeltene blir automatisk lagt inn i GUI når du begynner på oppgaven og fjerner `#define FOUR_A`.

b) **Legg til `TDT4102::TextInput` `personSeats` og `TDT4102::Button` `personNewBtn` i GUI.**

Du skal deklarere variablene i `.h`-filen, og initialisere de i `.cpp`-filen. Du slipper å legge knappen til GUI da dette skjer av seg selv likt som i forrige deloppgave. Funksjonaliteten til disse skal være å skrive inn et tall (som er større enn null) i `personSeats`, og trykke på knappen `personNewBtn` for å lage et `Car`-objekt med antall ledige plasser lik input i `personSeats`. Sjåføren må først ha plass, så du må også "reservere" et sete i `Car`-objektet! Dette skal du implementere i de kommende deloppgavene.

c) **Deklarer en `vector<Person*>` `people` som en medlemsvariabel, og så deklarer og implementer en ny funksjon, `void newPerson()`.**

Vektoren skal inneholde pekere til alle personer som blir lagt til gjennom tekstboksene.

`newPerson()` skal lese det som er skrevet inn i tekstboksene og legge til en ny person i vektoren med disse argumentene. Dette skal være et anonymt/navnløst objekt, så her må du bruke `new`:

```
people.emplace_back(new Person{/*Dine argument*/});
```

For å hente innholdet i tekstboksene, må du kalle medlemsfunksjonen `getText()` på innskrivings-feltene. Sjekk også om en av parameterene mangler, slik at det ikke legges til ufullstendige personer. Husk også å tømme tekstboksene hver gang du legger til en person, ved å kalle funksjonen `setText()`.

Tips: Du kan konvertere fra `string` som du får fra `getText()` på `personSeats`, til `int` ved hjelp av funksjonen `stoi("en streng")`. Den konverterer "en streng" til heltall. Dersom strengen ikke er et tall vil funksjonen gi en feilmelding. Det kan derfor være lurt å legge inn unntakshåndtering for tilfellene der brukeren ikke skriver inn et tall.

d) **Legg til funksjonen du nettopp implementerte, `newPerson()` som callback-funksjon til knappen `personNewBtn`.**

Se oppgave 4g for hvordan man legger til callback-funksjon.

e) **Test om programmet fungerer som forventet.**

En enkel måte å gjøre dette på er å lage en `public` funksjon som printer alle personene i `people`-vektoren. Denne funksjonen kan du kalle i `main()`.

7 Utvidelse av GUI (Frivillig)

I denne oppgaven kan du fullføre GUI-et for samkjøringen. Det skal nå være to sider: en for **Person** og en for **Meeting**. Man skal kunne se alle møter/personer som er lagt inn og kunne legge inn nye. Vinduet skal ha en knapp for å avslutte, to knapper som respektivt bytter til **Meeting**- og **Person**-siden, et tekstfelt for informasjon, et felt for hver parameter å legge inn, og to knapper som respektivt legger til en ny **Person** eller **Meeting**. I tillegg skal det være to felt der du kan velge en person å legge til et spesifikt møte, og en knapp som utfører dette.

- a) **Legg til en ny `TDT4102::TextInput`, `personData`, som privat medlem.**

Dette skal være et større tekstfelt der det er plass til flere linjer. Det skal fungere som display i vårt GUI, og skal vise personene som har blitt lagt inn.

- b) **Lag to nye funksjoner, `void showPersonPage()` og `void showMeetingPage()`.**

Disse skal vi bruke til å bytte mellom **Person** og **Meeting** sidene. For å gjøre dette, må `showPersonPage()` kalle medlemsfunksjonen `setVisible(true)` på alle elementer som er knyttet til den siden, mens `showMeetingPage()` må kalle `setVisible(false)`. Det omvendte gjelder for alle kommende elementer som er knyttet til **Meeting**-siden.

- c) **Legg til to `TDT4102::Buttons`, `createPersonButton`, og `createMeetingButton`, med tilhørende callback-funksjoner, `cb_persons()` og `cb_meetings()`.**

Callback-funksjonene skal respektivt kalle `showPersonPage()` og `showMeetingPage()`.

- d) **Legg inn fire nye `TDT4102::TextInput` til **Meeting**-sida: `meetingSubject`, `meetingDay`, `meetingStart` og `meetingEnd`.**

Disse skal vi senere bruke for å legge inn **Meeting**-objekt. Nå er det hurt å kalle `showPersonPage()` på slutten av **MeetingWindow**-konstruktøren.

- e) **Legg inn to `TDT4102::DropDownList` til **Meeting**-sida: `meetingLocation` og `meetingLeader`.**

`TDT4102::DropDownList` lager en rullegardinliste. For å legge til et valg må du kalle funksjonen `add` med en strengparameter. Legg inn de tre relevante stedene til `location` rullegardinlista `meetingLocation` vha. **MeetingWindow**-konstruktøren. Utvid `newPerson` til at navnet til den nye personen legges til som et valg i `meetingLeader`. Dette kan gjøres ved å bruke funksjonen til `TDT4102::DropDownList` som heter `setOption()`. Denne tar inn en parameter av typen `vector<string>` med alle valg som skal vises i rullegardinlista. Denne funksjonen vil oppdatere rullegardinlista til å vise valgene som står i `vector`-en.

- f) **Legg inn en ny funksjon, `void newMeeting()`, en callback som kaller denne, `cb_new_meeting()`, og en knapp med denne callbacken, `meetingNewBtn`.**

`newMeeting` skal lese parametere fra feltene og konstruere et **Meeting** i en ny `vector<Meeting*>`, `meetings`. `TDT4102::DropDownList` har medlemsfunksjonen `getValue()` som returnerer posisjonen til valget i lista, så bruk dette for å finne rett sted eller rett møteleder i vektoren over personer. Husk å sjekke om parameterene er gyldige.

- g) **Lag et display. `meetingData`** Dette displayet skal vise alle møtene som er lagt til i GUI.